# Chapter 4: Application Design Concepts and Principles

## Chapter Objectives

1. Explain and demonstrate the main advantages of an object ori- ented approach to system design including the effect of encap- sulation, inheritance, delegation, and the use of interfaces, on architectural issues.
2. Describe how the principle of separation of concerns has been applied to the main system design
3. Describe and apply software design Concepts to system designs.

# Fundamentals of Object Oriented Programming Concepts

There are three major features in object-oriented programming: encapsulation, inheritance and polymorphism.

## Encapsulation

**Encapsulation enforces modularity.**

Encapsulation refers to the creation of self-contained modules that bind processing func- tions to the data. These user-defined data types are called classes, and one instance of a class is an object. In other words encapsulation means that the attributes (data) and the behaviors (code) are encapsulated in to a single object.

In other models, namely a structured model, code is in files that are separate from the data. An object, conceptually, combines the code and data into a single entity.

In programing at the class level, Encapsulation is the technique of making the fields in a class private and providing access to the fields via public methods. If a field is declared private, it cannot be accessed by anyone outside the class, thereby hiding the fields within the class.

For this reason, encapsulation is also referred to as data hiding. Encapsulation can be described as a protective barrier that prevents the code and data being randomly accessed by other code defined outside the class.

Access to the data and code is tightly controlled by an interface. The main benefit of encapsulation is the ability to modify our implemented code without breaking the code of others who use our code. With this feature Encapsulation gives maintainability, flexibility and extensibility to our code.

The code below shows a class which is encapsulated.

Benefits of Encapsulation include:

1. The fields of a class can be made read-only or write-only.
2. A class can have total control over what is stored in its fields.
3. The users of a class do not know how the class stores its data. A class can change the data type of a field, and users of the class do not need to change any of their code.

## Inheritance

**Inheritance passes knowledge down.**

Inheritance can be defined as the process where one object acquires the properties of another. With the use of inheritance the information is made manageable in a hierarchical order. When we talk about inheritance the most commonly used keyword would be extends and implements. These words would determine whether one object **IS-A** type of another.

By using these keywords we can make one object acquire the properties of another object.

Usually classes are created in hierarchies, and inheritance allows the structure and methods in one class to be passed down the hierarchy. That means less programming is required when adding functions to complex systems.

If a step is added at the bottom of a hierarchy, then only the processing and data associated with that unique step needs to be added. Everything else about that step is inherited. The ability to reuse existing objects is considered a major advantage of object technology.

One of the major design issues in **O-O** programming is to factor out commonality of the various classes.

For example, say you have a **Dog class** and a **Cat class**, and each will have an attribute for eye color. In a procedural model, the code for Dog and Cat would each contain this attribute. In an **O-O design**, the color attribute can be abstracted up to a class called Mammal along with any other common attributes and methods.

Lets revisit Encapsulation: One of the primary advantages of using objects is that the object need not reveal all of its attributes and behaviors.

In good **O-O design** , an object should only reveal the interfaces needed to interact with it. Details not important to the use of the object should be hidden from other objects. This is called encapsulation. The interface is the fundamental means of communication between objects. Each class design specifies the interfaces for the proper instantiation and operation of objects. Any behavior that the object provides must be invoked by a message sent using one of the provided interfaces. The interface should completely describe how users of the class interact with the class.

In Java, the methods that are part of the interface are designated as public; everything else is part of the private implementation.

This is The Encapsulation Rule: Whenever the interface/implementation paradigm is covered, you are really talking about encapsulation. The basic question is what in a class should be exposed and what should not be exposed. This encapsulation pertains equally to data and behavior.

When talking about a class, the primary design decision revolves around encapsulating both the data and the behavior into a well-written class. In other words the process of packaging your program, dividing each of its classes into two distinct parts: the interface and the implementation is Encapsulation in the big picture.

Encapsulation is so crucial to O-O development that it is one of the generally accepted object-oriented designs cardinal rules.

Yet, Inheritance is also considered one of the four primary O-O concepts. However, in one way, inheritance actually breaks encapsulation!

How can this be? Is it possible that two of the three primary concepts of O-O are incompatible with each other?

There are three criteria that determine whether or not a language is object-oriented: encapsulation, inheritance, polymorphism.

To be considered a true object-oriented language, the language must support all three of these concepts.

Because encapsulation is the primary object-oriented mandate, you must make all at- tributes private. Making the attributes public is not an option.

Thus, it appears that the use of inheritance may be severely limited. If a subclass does not have access to the attributes of its parent, this situation presents a sticky design problem. To allow subclasses to access the attributes of the parent, language designers have included the access modifier protected. There are actually two types of protected access.

Rule of thumb is **AVOID CONCRETE INHERITANCE!!!** Only use it when it has been imposed on your and you

cannot change the class you are reusing. What is the Inheritance alternative? **Favour Composition over inheritance.**

# Polymorphism

Polymorphism is the ability of an object to take on many forms. The most common use of polymorphism in OOP occurs when a parent class reference is used to refer to a child class object. Any java object that can pass more than one IS-A test is considered to be polymorphic. In Java, all java objects are polymorphic since any object will pass the IS-A test for their own type and for the class Object. It is important to know that the only possible way to access an object is through a reference variable.

A reference variable can be of only one type. Once declared the type of a reference variable cannot be changed. The reference variable can be reassigned to other objects provided that it is not declared final.

The type of the reference variable would determine the methods that it can invoke on the object. A reference variable can refer to any object of its declared type or any subtype of its declared type. A reference variable can be declared as a class or interface type.

Let us look at an example.

Now the Cow class is considered to be polymorphic since this has multiple inheritance. Following are true for the above example:

- Cow IS-A aAnimal
- Cow IS-A Vegetarian
- Cow IS-A Cow
- Cow IS-A Object

```
public interface Vegetarian {}
public class Animal {}
public class Cow extends Animal implements Vegetarian{}
```

```
Cow c = new Cow();
Animal a = c;
Vegetarian v = c;
Object o = c;
```

All the reference variables **c,a,v,o** refer to the same Cow object in the heap.

# Design Principles

Great software is built in an agile way. Agility is about building software in tiny incre- ments, it is important to take the time to ensure that the software has a good structure that is flexible, maintainable, and reusable.

In an agile team, the big picture evolves along with the software. With each iteration, the team improves the design of the system so that it is as good as it can be for the system as it is now.

The team does not spend very much time looking ahead to future requirements and needs. Nor does it try to build in today the infrastructure to support the features that may be needed tomorrow.

Rather, the team focuses on the current structure of the system, making it as good as it can be. This is a way to incrementally evolve the most appropriate architecture and design for the system. It is also a way to keep that design and architecture appropriate as the system grows and evolves over time.

Agile development makes the process of design and architecture continuous. How do we know how whether the design of a software system is good? Below are the tips to help us detect bad design

The symptoms are:

1. **Rigidity:** The design is difficult to change.Rigidity is the tendency for software to be difficult to change. A design is rigid if a single change causes a cascade of subsequent changes in dependent modules.
2. **Fragility** is the tendency of a program to break in many places when a single change is made.
3. **Immobility:** A design is immobile when it contains parts that could be useful in other systems, but the effort and risk involved with separating those parts from the original system are too great.
4. **Viscosity.** It is difficult to do the right thing. Viscosity comes in two forms: viscosity of the software and viscosity of the environment. Goal is to design our software such that the changes that preserve the design are easy to make. Viscos- ity of environment comes about when the development environment is slow and inefficient In both cases, a viscous project is one in which the design of the software is difficult to preserve. We want to create systems and project environments that make it easy to preserve and improve the design.
5. **Needless complexity.** Overdesign. A design smells of needless complexity when it contains elements that aren't currently useful. This frequently happens when developers anticipate changes to the requirements and put facilities in the software to deal with those potential changes.
6. **Needless repetition.** Mouse abuse.Cut and paste may be useful text-editing operations, but they can be disastrous code-editing operations. Bad code can easily propagate to all modules and make it hard to change.
7. **Opacity.** Disorganized expression. Opacity is the tendency of a module to be difficult to understand. Code can be written in a clear and expressive manner, or it can be written in an opaque and convoluted manner. Code that evolves over time tends to become more and more opaque with age. A constant effort to keep the code clear and expressive is required in order to keep opacity to a minimum.

# Software Design Principles

## The Single-Responsibility Principle (SRP)

The Single Responsibility Principle (SRP) states that a class or module should have one, and only one, reason to change.

In the context of the SRP, we define a responsibility to be a reason for change. If you can think of more than one motive for changing a class, that class has more than one responsibility

If a class assumes more than one responsibility, that class will have more than one reason to change. Why SRP matters

1. We want it to be easy to reuse code
2. Big classes are more difficult to change
3. Big classes are harder to read
4. Dont code for situations that you wont ever need
5. Dont create unneeded complexity
6. However, **more class files != more complicated** Offshoot of SRP - Small Methods
7. A method should have one purpose (reason to change)
8. Easier to read and write, which means you are less likely to write bugs
9. Write out the steps of a method using plain English method names

## The Open/Closed Principle (OCP)

Software entities (classes, modules, functions, etc.) should be open for ex- tension but closed for modification.

When a single change to a program results in a cascade of changes to dependent modules, the design smells of rigidity. OCP advises us to refactor the system so that further changes of that kind will not cause more modifications.

If OCP is applied well, further changes of that kind are achieved by adding new code, not by changing old code that already works.

Anytime you change code, you have the potential to break it

Modules that conform to OCP have two primary attributes.

1. They are open for extension. This means that the behavior of the module can be extended. As the requirements of the application change, we can extend the module with new behaviors that satisfy those changes. We are able to change what the module does.
2. They are closed for modification. Extending the behavior of a module does not result in changes to the source, or binary, code of the module.

The normal way to extend the behavior of a module is to make changes to the source code of that module. Modules behaviours can be changed without modifying the code by means of abstractions. In object-oriented programming language (OOPL), it is possible to create abstractions that are fixed and yet represent an unbounded group of possible behaviors.

The abstractions are abstract base classes, and the unbounded group of possible behav- iors are represented by all the possible derivative classes. It is possible for a module to manipulate an abstraction.

Such a module can be closed for modification, since it depends on an abstraction that is fixed. Yet the behavior of

that module can be extended by creating new derivatives of the abstraction It is possible for a module to manipulate an abstraction. Such a module can be closed for modification, since it depends on an abstraction that is fixed. Yet the behavior of that module can be extended by creating new derivatives of the abstraction. Two Design Patterns used to enforce OCP are Template Method and Strategy.

These two patterns are the most common ways of satisfying OCP. They represent a clear sep- aration of generic functionality from the detailed implementation of that functionality. Will cover more of these in the next chapter on Design Patterns and Refactoring.

**The Open/Closed Principle is at the heart of object-oriented design. Con- formance to this principle is what yields the greatest benefits claimed for object-oriented technology: flexibility, reusability, and maintainability.**

# The Liskov Substitution Principle (LSP)

**Subclasses should be substitutable for their base classes. The Super class and the sub class should be substitutable and make sense when used.**

We mentioned that OCP is the most important of the class category principles. We can think of the Liskov Substitution Principle (LSP) as an extension to OCP. In order to take advantage of LSP, we must adhere to OCP because violations of LSP also are violations of OCP, but not vice versa.

In its simplest form, LSP is difficult to differentiate from OCP, but a subtle difference does exist. OCP is centered around abstract coupling. LSP, while also heavily dependent on abstract coupling, is in addition heavily dependent on preconditions and postcondi- tions, which is LSPs relation to Design by Contract, where the concept of preconditions and postconditions was formalized.

A precondition is a contract that must be satisfied before a method can be invoked. A postcondition, on the other hand, must be true upon method completion. If the precondition is not met, the method shouldnt be invoked, and if the postcondition is not met, the method shouldnt return.

The Liskovs Substitution Principle provides a guideline to sub-typing any existing type. Stated formally it reads:

***If for each object o1 of type S there is an object o2 of type T such that for all programs P defined in terms of T, the behaviour of P is unchanged when o1 is substituted for o2 then S is a subtype of T.***

In a simpler term, if a program module is using the reference of a Base class, then it should be able to replace the Base class with a Derived class without affecting the functioning of the program module.

# The Dependency-Inversion Principle (DIP)

**Depend upon abstractions. Do not depend upon concretions.**

The Dependency Inversion Principle (DIP) formalizes the concept of abstract coupling and clearly states that we should couple at the abstract level, not at the concrete level.

Two classes are tightly coupled if they are linked together and are dependent on each other Tightly coupled classes can not work independent of each other It Makes changing one class difficult because it could launch a wave of changes through tightly coupled classes

In our own designs, attempting to couple at the abstract level can seem like overkill at times. Pragmatically, we should apply this principle in any situation where were unsure whether the implementation of a class may change in the future.

But in reality, we encounter situations during development where we know exactly what needs to be done.

Requirements state this very clearly, and the probability of change or extension is quite low. In these situations, adherence to DIP may be more work than the benefit realized.

At this point, there exists a striking similarity between DIP and OCP. In fact, these two principles are closely related. Fundamentally, DIP tells us how we can adhere to OCP. Or, stated differently, if OCP is the desired end, DIP is the means through which we achieve that end. While this statement may seem obvious, we commonly violate DIP in a certain situation and dont even realize it.

Abstract coupling is the notion that a class is not coupled to another concrete class or class that can be instantiated. Instead, the class is coupled to other base, or abstract, classes. This abstract class can be either a class with the abstract modifier or a Java interface data type.

Regardless, this concept actually is the means through which LSP achieves its flexibility, the mechanism required for DIP, and the heart of OCP. The OCP is the guiding principle for good Object- Oriented design.

The design typically has two aspects with it. One, you design an application module to make it work and second, you need to take care whether your design, and thereby your application, module is reusable, flexible and robust. The OCP tells you that the software module should be open for extension but closed for modification.

As you might have already started thinking, this is a very high level statement and the real problem is how to achieve this. Well, it comes through practice, experience and constant inspection of any piece of design, understanding that how it performs and works tackles with the expanding re- quirements of the application. But even though you are a new designer as student, you can follow certain principles to make sure that your design is a good one.

The Dependency Inversion Principle helps you make your design OCP compliant. Formally stated, the principle makes two points: High level modules should not depend upon low level modules. Both should depend upon abstractions Abstractions should not depend upon details. Details should depend upon abstractions.

## The Interface Segregation Principle (ISP)

Many specific interfaces are better than a single, general interface. This principle deals with the disadvantages of ”fat” interfaces. Classes whose interfaces are not cohesive have ”fat” interfaces. In other words, the interfaces of the class can be broken up into groups of methods. Each group serves a different set of clients. Thus, some clients use one group of methods, and other clients use the other groups.

Any interface we define should be highly cohesive. In Java, we know that an interface is a reference data type that can have method declarations, but no implementation. In essence, an interface is an abstract class with all abstract methods.

As we define our interfaces, it becomes important that we clearly understand the role the interface plays within the context of our application. In fact, interfaces provide flexibility: They allow objects to assume the data type of the interface.

Consequently, an interface is simply a role that an object plays at some point throughout its lifetime. It follows, rather logically, that when defining the operation on an interface, we should do so in a manner that doesnt accommodate multiple roles.

Therefore, an interface should be responsible for allowing an object to assume a SINGLE ROLE, assuming the class of which that object is an instance implements that interface. **Composite Reuse Principle (CRP)**

Favor polymorphic composition of objects over inheritance. The **Composite Reuse Principle (CRP)** prevents us from making one of the most catas- trophic mistakes that contribute to the demise of an object-oriented system: using in- heritance as the primary reuse mechanism.

Inheritance can be thought of as a generalization over a specialization relationship That is, a class higher in the

inheritance hierarchy is a more general version of those inherited from it. In other words, any ancestor class is a partial descriptor that should define some default characteristics that are applicable to any class inherited from it.

Any time we have to override default behavior defined in an ancestor class, we are saying that the ancestor class is not a more general version of all of its descendants but actually contains descriptor characteristics that make it too specialized to serve as the ancestor of the class in question.

Therefore, if we choose to define default behavior on an ancestor, it should be general enough to apply to all of its descendents. In practice, its not uncommon to define a default behavior in an ancestor class. However, we should still accommodate CRP in our relationships.

# Principle of Least Knowledge (PLK)

For an operation O on a class C, only operations on the following objects should be called:itself, its parameters, objects it creates, or its contained instance objects.

The Principle of Least Knowledge (PLK) is also known as the Law of Demeter. The basic idea is to avoid calling any methods on an object where the reference to that object is obtained by calling a method on another object.

Instead, this principle recommends we call methods on the containing object, not to obtain a reference to some other object, but instead to allow the containing object to forward the request to the object we would have formerly obtained a reference to.

The primary benefit is that the calling method doesnt need to understand the structural makeup of the object its invoking methods upon. The obvious disadvantage associated with PLK is that we must create many methods that only forward method calls to the containing classes internal components. This can contribute to a large and cumbersome public interface.

An alternative to PLK, or a variation on its implementation, is to obtain a reference to an object via a method call, with the restriction that any time this is done, the type of the reference obtained is always an interface data type.

This is more flexible because we arent binding ourselves directly to the concrete imple- mentation of a complex object, but instead are dependent only on the abstractions of which the complex object is composed.

This is how many classes in Java typically resolve this situation.

# Software Packaging Principles

In this section, we explore the principles of design that help us split a large software system into packages.

As software applications grow in size and complexity, they require some kind of highlevel organization. Classes are convenient unit for organizing small applications but are too finely grained to be used as the sole organizational unit for large applications. Something ”larger” than a class is needed to help organize large applications. That something is called a package, or a component.

This section outlines six principles for managing the contents and relationships between components. The first three, principles of package cohesion, help us allocate classes to packages. The last three principles govern package coupling and help us determine how packages should be interrelated.

The last two principles also describe a set of dependency management metrics that allow developers to measure and characterize the dependency structure of their designs.

# Principles of Component Cohesion: Granularity

The principles of component cohesion help developers decide how to partition classes into components. These principles depend on the fact that at least some of the classes and their interrelationships have been discovered. Thus, these principles take a bottom-up view of partitioning.

## The Reuse/Release Equivalence Principle (REP)

**The granule of reuse is the granule of release.**

REP states that the granule of reuse, a component, can be no smaller than the granule of release. Anything that we reuse must also be released and tracked. It is not realistic for a developer to simply write a class and then claim that it is reusable.

Reusability comes only after a tracking system is in place and offers the guarantees of notification, safety, and support that the potential reusers will need.

Whenever a client class wishes to use the services of another class, we must reference the class offering the desired services. If the class offering the service is in the same package as the client, we can reference that class using the simple name. If, however, the service class is in a different package, then any references to that class must be done using the class fully qualified name, which includes the name of the package. Any Java class may reside in only a single package.

Therefore, if a client wishes to utilize the services of a class, not only must we reference the class, but we must also explicitly make reference to the containing package. Failure to do so results in compile- time errors. Therefore, to deploy any class, we must be sure the containing package is deployed. Because the package is deployed, we can utilize the services offered by any public class within the package.

While we may presently need the services of only a single class in the containing package, the services of all classes are available to us.

Consequently, our unit of release is our unit of reuse, resulting in the Release Reuse Equivalency Principle (REP). This leads us to the basis for this principle, and it should now be apparent that the packages into which classes are placed have a tremendous impact on reuse. Careful consideration must be given to the allocation of classes to packages.

## The Common Reuse Principle (CReP)

**Classes that arent reused together should not be grouped together.**

If we need the services offered by a class, we must import the package containing the necessary classes.

As we stated previously in our discussion of REP (Release Reuse Equivalency Principle), when we import a package, we also may utilize the services offered by any public class within the package. In addition, changing the behavior of any class within the service package has the potential to break the client.

Even if the client doesnt directly reference the modified class in the service package, other classes in the service package being used by clients may reference the modified class. This creates indirect dependencies between the client and the modified class that can be the cause of mysterious behavior. We can state the following: If a class is dependent on another class in a different package, then it is dependent on all classes in that package, albeit indirectly. This principle has a negative connotation.

It doesnt hold true that classes that are reused together should reside together, depend- ing on CCP. Even though classes may always be reused together, they may not always change together. In striving to adhere to CCP, separating a set of classes based on their likelihood to change together should be given careful consideration.

Of course, this impacts REP because now multiple packages must be deployed to use this functionality. Experience tells us that adhering to one of these principles may impact the ability to adhere to another. Whereas REP and Common Reuse Principle (CReP) emphasize reuse, CCP emphasizes maintenance.

## The Common Closure Principle (CCP)

**Classes that change together, belong together.**

The basis for the Common Closure Principle (CCP) is rather simple. Adhering to fundamental programming best practices should take place throughout the entire system. Functional cohesion emphasizes well-written methods that are more easily maintained. Class cohesion stresses the importance of creating classes that are functionally sound and dont cross responsibility boundaries.

And package cohesion focuses on the classes within each package, emphasizing the overall services offered by entire packages.

During development, when a change to one class may dictate changes to another class, its preferred that these two classes be placed in the same package.

Conceptually, CCP may be easy to understand; however, applying it can be difficult because the only way that we can group classes together in this manner is when we can predictably determine the changes that might occur and the effect that those changes might have on any dependent classes.

Predictions often are incorrect or arent ever realized.

Regardless, placement of classes into respective packages should be a conscious decision that is driven not only by the relationships between classes, but also by the cohesive nature of a set of classes working together.

# Principles of Component Coupling: Stability

The next three principles deal with the relationships between components. Here again, we will run into the tension between developability and logical design. The forces that impinge on the architecture of a component structure are technical, political, and volatile.

## The Acyclic Dependencies Principle (ADP)

The dependencies between packages must form no cycles.

Cycles among dependencies of the packages composing an application should almost always be avoided.

Packages should form a Directed Acyclic Graph (DAG). Acyclic Dependencies Principle (ADP) is important from a deployment perspective.

Along with packages being reusable and maintanable, the should also be deployable as well. Just as in the class design, the package design should have defined dependencies so that it is deployment- friendly.

## The Stable-Dependencies Principle (SDP)

**Depend in the direction of stability.**

Stability implies that an item is fixed, permanent, and unvarying. Attempting to change an item that is stable is more difficult than inflicting change on an item in a less stable state. Aside from poorly written code, the degree of coupling to other packages has a dramatic impact on the ease of change.

Those packages with many incoming dependencies have many other components in our application dependent on them.

These more stable packages are difficult to change because of the far-reaching conse- quences the change may have throughout all other dependent packages. On the other hand, packages with few incoming dependencies are easier to change.

Those packages with few incoming dependencies most likely will have more outgoing dependencies. A package with no incoming or outgoing dependencies is useless and isnt part of an application because it has no relationships.

Therefore, packages with fewer incoming, and more outgoing dependencies, are less stable. Stability metrics Stability is calculated by counting the number of dependencies that enter and leave that component.

These counts allow us to calculate the positional stability of the component:

**Ca (afferent couplings):** The number of classes outside this component that depend on classes within this component

**Ce (efferent couplings):** The number of classes inside this component that depend on classes outside this component

**Formula: I(instability )** = Ce/(Ce +Ca)

This metric has the range [0,1]. I = 0 indicates a maximally stable component. I = 1 indicates a maximally unstable component.

The Ca and Ce metrics are calculated by counting the number of classes outside the component in question that have dependencies on the classes inside the component in question.

## The Stable-Abstractions Principle (SAP)

Stable packages should be abstract packages.

One of the greatest benefits of object orientation is the ability to easily maintain our systems. The high degree of resiliency and maintainability is achieved through abstract coupling. By coupling concrete classes to abstract

classes, we can extend these abstract classes and provide new system functions without having to modify existing system structure.

Consequently, the means through which we can depend in the direction of stability, and help ensure that these more depended-upon packages exhibit a higher degree of stability, is to place abstract classes, or interfaces, in the more stable packages.

We can state the following: More stable packages, containing a higher number of abstract classes, or interfaces, should be heavily depended upon. Less stable packages, containing a higher number of concrete classes, should not be heavily depended upon. Any packages containing all abstract classes with no incoming dependencies are utterly useless.

On the other hand, packages containing all concrete classes with many incoming depen- dencies are extremely difficult to maintain.

**Measuring abstraction**

The A metric is a measure of the abstractness of a component. Its value is simply the ratio of abstract classes in a component to the total number of classes in the component, where **Nc** is the number of classes in the component. **Na** is the number of abstract classes in the component. Remember, an abstract class is a class with at least one abstract method and cannot be instantiated:

# Chapter Exercises

1. Write small program using TDD method to demonstrate the three core principals of Object Oriented Programming
2. In question 1, you wrote code demonstrating inheritance. Modify your code to use an alternative solution to inheritance.
3. There are 7 core software principles and for each write code that violet the principle and then write code that obeys the principle
4. Write code that violets ADP and also write code that corrects the violation.